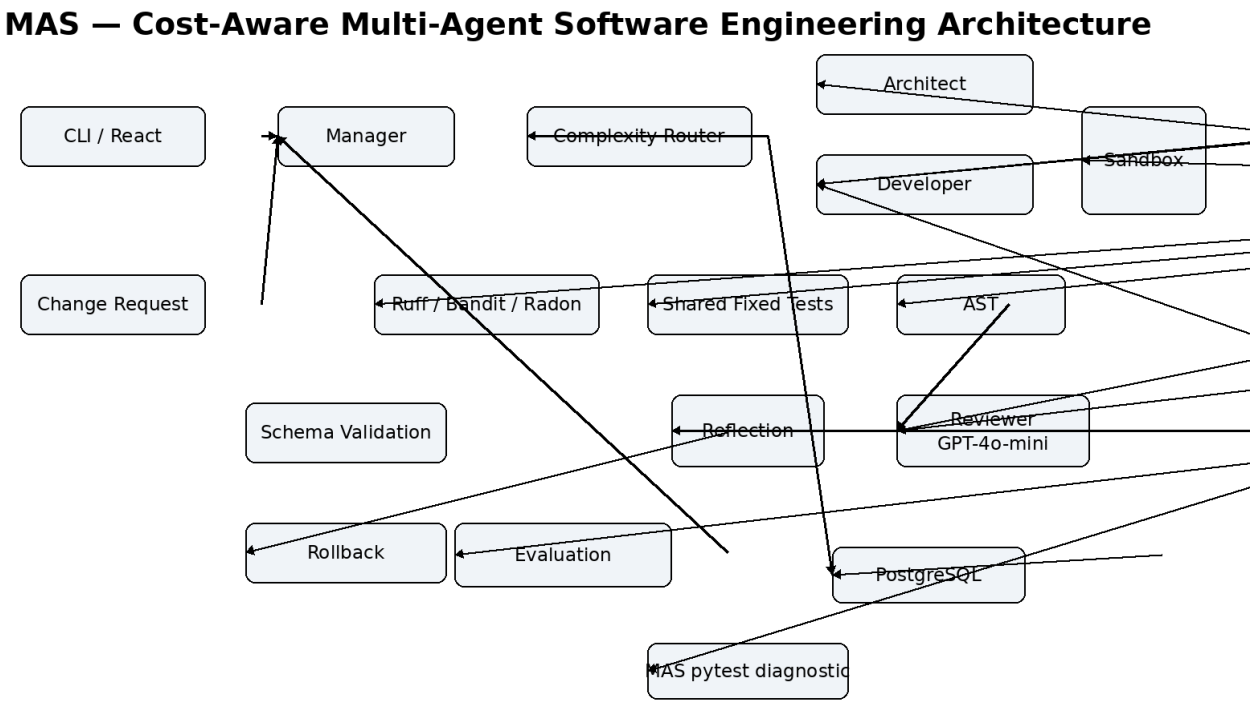
MAS — Multi-Agent Software Engineering Team

7-Day MVP Research Documentation — Revised

Five-document specification with requested evaluation, routing, security, stopping and ticket revisions.



MAS Multi-Agent Software Engineering Team

7-Day MVP Research Documentation — Revised

Product Requirements • Technical Architecture • Security & Access • Frontend • Feature Tickets Reference architecture: You et al., “iCoder: A multi agent software development platform,” Information and Software Technology 198 (2026), 108224. Honest comparison rule: MAS is compared directly with ChatDev; iCoder is referenced only through published results.

Contents

Document 1 — Product Requirements Document (PRD)

Document 2 — Technical Architecture

Document 3 — Security and Access Specification

Document 4 — Frontend Specification

Document 5 — Feature Ticket List

Research positioning, evaluation protocol, limitations and changelog

Assumptions

MVP generation language is Python, consistent with the iCoder experimental setup.

Direct empirical comparator is ChatDev; iCoder source code is unavailable.

Main comparison uses GPT-4o. The cheaper model is explicitly GPT-4o-mini.

For iCoder-style consistency, MAS uses the lightweight sentence-transformers model `all-MiniLM-L6-v2` as an implementation choice. This is not claimed to be the embedding model used by iCoder; the paper specifies only an embedding function.

Model name/version, temperature, seed, embedding model, library versions and Docker image tag are recorded for every run.

Token counts for both systems come only from provider API usage fields; local estimates are not used as measured token counts.

DOCUMENT 1 — PRODUCT REQUIREMENTS DOCUMENT

1. Problem

LLM-based multi-agent software generation can improve system-level code generation through decomposition, architecture constraints and iterative review, but these mechanisms can introduce substantial computational overhead. iCoder separates requirement analysis, architecture design, development and quality assurance, and uses Sprints as the iterative control unit. Its RABot decomposes requirements and creates stories/Sprints; UMLBot creates architectural views; DevBot generates code; ReviewBot performs dependency-aware review and testing. (iCoder, Secs. 3.1–3.3; Figs. 1–7.)

The research problem for MAS is: Can a multi-agent software-engineering workflow retain the quality benefits of structured development while reducing unnecessary model calls, context size and iteration cost?

2. Goals

3. Non-goals

4. Functional Requirements

5. Success Metrics

Metric note. iCoder defines Quality as Completeness × Executability × Consistency and reports arithmetic mean Quality over the evaluation set (Sec. 4.3, Tables 1–2). The paper's printed executability equation is internally inconsistent with its surrounding prose; MAS uses successful executions / evaluated projects as the operational execution-pass-rate metric and documents this interpretation.

6. Gap → Feature Traceability

7. Seven-Day Roadmap

DOCUMENT 2 — TECHNICAL ARCHITECTURE

1. System Architecture

Figure 1. MAS architecture. MAS-generated pytest remains a diagnostic path and is not the main cross-system quality score.

Mermaid Diagram Source

flowchart TD U[CLI / React] --> M[Manager] M --> R[Complexity Router] R -->|Simple| D[Developer] R -->|Structured| A[Architect] A --> D D --> S[Sandbox Workspace] S --> X[AST Dependency Analyzer] S --> ST[Shared Fixed Test Harness] S --> Q[Ruff / Bandit / Radon] S --> PT[MAS pytest Diagnostic] X --> RV[Reviewer - GPT-4o-mini] ST --> RV Q --> RV RV --> RF[Reflection] RF -->|continue| D RF -->|stop| E[Evaluation / Result] CR[Requirement Change] --> M F[Artifact Fault] --> V[Schema Validator] V -->|invalid| RB[Rollback] RB --> M P[(PostgreSQL)] --- M P --- E

The design preserves iCoder's requirements → architecture → development → iteration loop while making expensive stages conditional. iCoder describes four core agents — RABot, UMLBot, DevBot and ReviewBot — coordinated by Sprint iteration (Secs. 3.1–3.3; Figs. 1–7).

2. Per-Agent Role, Inputs and Outputs

3. LangGraph State Schema

```
class MASState(TypedDict):
    run_id: str
    task_id: str
    requirement: str

    complexity: str
```

```
    complexity_score: int
    router_features: dict

    architecture: dict | None
    sprint_plan: dict | None

    repo_snapshot: str
    changed_files: list[str]
    relevant_uml: list[str]

    artifacts: dict
    validation_errors: list[str]

    ast_report: dict
    shared_test_report: dict
    mas_pytest_report: dict
    quality_report: dict
    review_report: dict

    iteration: int
    previous_adaptive_score: float
    marginal_gain: float
    stop_reason: str | None

    input_tokens: int
    output_tokens: int
    total_tokens: int
    estimated_cost: float
```

4. JSON Schemas for Inter-Agent Artifacts

{ "$id": "UserStory", "type": "object", "required": ["id", "role", "action", "goal", "acceptance_criteria"], "properties": { "id": {"type": "string"}, "role": {"type": "string"}, "action": {"type": "string"}, "goal": {"type": "string"}, "acceptance_criteria": { "type": "array", "items": {"type": "string"} }, "priority": {"enum": ["low", "medium", "high"]}, "estimate": {"type": "integer", "minimum": 1} } }

{ "$id": "CodePatch", "type": "object", "required": ["files", "summary", "tests_expected"], "properties": { "files": {"type": "array", "items": {"type": "string"}}, "summary": {"type": "string"}, "tests_expected": {"type": "array", "items": {"type": "string"}} } }

Required artifact types: TaskPlan, UserStory, Architecture, CodePatch, ReviewReport, TestReport and ReflectionDecision. Pydantic models are canonical; JSON Schema is exported for runtime validation.

5. Concrete Router Decision Logic

The Router is deterministic and does not call an LLM. Each feature is a keyword/regex rule applied to the normalized requirement text. Multiple matches in one feature count according to the rule below.

Threshold: score 0–3 → SIMPLE; score 4+ → STRUCTURED. A 10-task hand-labeled router-accuracy check is run separately and reports accuracy, false-SIMPLE rate and false-STRUCTURED rate. The hand labels are fixed before evaluation.

Note: these deterministic features are a concrete MVP routing policy, not a claim that the iCoder paper used the same rules.

6. AST Review Pipeline

Changed .py files ↓ ast.parse() ↓ Import / ImportFrom extraction ↓ Module resolution ↓ Directed dependency graph ↓ Cycle detection ↓ Undefined/local-import checks ↓ Changed-module impact report

Structural checks are deterministic and do not consume model tokens. GPT-4o-mini is reserved for semantic/requirement review. This is an MVP reinterpretation of iCoder's R-DCGG review role (Sec. 3.2.4).

7. Neutral Cross-System Quality Checks

iCoder-style completeness: proportion of implemented functions after the documented placeholder rule.

iCoder-style executability: successful execution/pass rate using the same evaluator environment.

iCoder-style consistency: cosine similarity between requirement and generated code embeddings using `all-MiniLM-L6-v2`; this is an MAS implementation choice, not a claimed reproduction of iCoder's embedding model.

iCoder-style Quality: arithmetic mean of per-project Completeness × Executability × Consistency.

Ruff, Bandit and Radon are run with identical commands/settings on MAS and ChatDev outputs.

Shared fixed test method: a neutral black-box evaluator protocol is fixed before the comparison and applied identically to both systems. It may use task-specific fixed inputs/expected outputs, but it never uses tests generated by either system.

MAS-generated pytest pass rate and coverage are collected separately as MAS-only diagnostics.

8. Adaptive Stopping Rule

The adaptive score is separate from the iCoder-style Quality metric. Default weights:

CE_t = 0.5 * (Completeness_t + Executability_t) S_t = 0.30*SharedTest_t + 0.10*Ruff_t + 0.15*Bandit_t + 0.10*Radon_t + 0.15*Reviewer_t + 0.20*CE_t Δ_t = S_t - S_(t-1) Defaults: STOP_EPSILON = 0.02 PATIENCE = 2 MAX_ITERATIONS = 3

Stop when: (1) no critical defects and the shared fixed test passes; OR (2) Δ remains below 0.02 for two consecutive iterations; OR (3) three iterations are reached. All weights, STOP_EPSILON, PATIENCE and MAX_ITERATIONS are configurable in one versioned configuration file.

9. Change-Request and Fault-Recovery Flow

CHANGE REQUEST new requirement → requirement diff → AST/import impact analysis → identify affected modules → update only affected files → update relevant UML only if architectural impact exists → shared regression test → obsolete-code detection → final neutral evaluation FAULT RECOVERY agent output → JSON Schema validation → valid? yes → next agent → invalid → retry producer once → still invalid → rollback last valid checkpoint → re-run

producer → record detection/recovery metrics

10. PostgreSQL Tables

11. FastAPI Endpoints

POST /runs GET /runs/{id} GET /runs/{id}/results GET /runs/{id}/events POST /runs/{id}/change POST /runs/{id}/cancel WS /ws/runs/{id} GET /evaluations/{id}

Authentication endpoints are intentionally absent in the local single-user MVP; authentication/roles are future work.

12. Model Abstraction and Reproducibility

class LLMProvider: async def generate(self, messages, model, response_schema=None, temperature=0) -> ModelResult: ... ModelResult: text model_name provider_model_version input_tokens output_tokens total_tokens latency_ms request_id estimated_cost

Every run records model name/version, temperature, seed, embedding model, Python/package library versions, Docker image tag and benchmark configuration. For MAS and ChatDev, measured token counts are taken only from the provider API usage fields.

13. Evaluation Harness

eval/ tasks.json srdd_loader.py runner.py chatdev_adapter.py scorers/ icoder_metrics.py ruff.py bandit.py radon.py shared_test.py mas_pytest_diagnostic.py stats.py reports.py config.yaml

The SRDD loader downloads the dataset, selects four tasks per category using a fixed seed, and saves the exact selected tasks to tasks.json. Categories: Education, Work, Life, Games, Creation.

Main comparison: 20 tasks × 3 repeats × 2 systems on GPT-4o = 120 model runs. MAS-only cheaper-model arm: 20 tasks × 3 repeats on GPT-4o-mini = 60 model runs. Ablations: 10 tasks × 2 repeats × 6 configurations on GPT-4o-mini = 120 model runs. Main + cheaper-model + ablations = 300 model runs.

Auxiliary diagnostics do not expand the 300-run core benchmark: router accuracy uses 10 hand-labeled requirement texts; the change-request mini-test uses 6 tasks with one injected change each; fault injection uses F1 malformed UML and F2 incomplete user story and is applied to selected runs.

Planning token budget (engineering cap, not a benchmark result): 40,000 tokens/run for GPT-4o conditions and 20,000 tokens/run for GPT-4o-mini conditions gives an upper planning envelope of 8.4 million tokens across the 300 core model runs. Actual reported usage must come only from provider API usage fields. If the team prefers, replace these caps with a one-task pilot mean before launching the full batch.

Primary statistical unit: task-level paired comparison across the three repeats. Primary significance test: two-sided paired permutation test, α=0.05. Report mean ± SD, median, effect size, p-value and confidence interval.

14. Comparison to iCoder

DOCUMENT 3 — SECURITY AND ACCESS SPECIFICATION

1. Security Objective

The MVP is local and single-user. It does not implement JWT login, roles, ownership checks, enterprise access control or multi-user audit identity. Security focuses on protecting the host, secrets, execution environment and agent workflow.

2. Threat Model

3. Sandboxed Execution

`--network none`

1 CPU core

512 MB memory target

Limited PIDs

30–60 second execution timeout

Read-only base filesystem

Writable `/workspace` only

No host Docker socket

No host filesystem mounts except the isolated workspace

Disposable container per execution

External package installation is disabled during evaluation; dependencies come from a predefined evaluation image.

4. Prompt-Injection Handling

SYSTEM POLICY ↓ TASK METADATA ↓ USER REQUIREMENT ↓ AGENT INSTRUCTIONS

User requirements never override system instructions.

Agent artifacts are data, not instructions.

JSON artifacts are parsed structurally.

Tool calls are restricted to known application functions.

Generated code never receives API secrets.

Inter-agent communication uses structured conclusions rather than chain-of-thought.

5. Secret Handling

OPENAI_API_KEY= DATABASE_URL=

Store secrets in environment variables or a local secret file excluded by `.gitignore`.

Never write secrets to generated repositories, artifacts, logs or evaluation output.

Provide `.env.example`, not a committed `.env`.

Redact common key/token patterns from event summaries.

6. Access Model

MVP: single local user. No JWT, no roles, no ownership checks, no users table and no audit_logs table.

Future work: authentication, role-based access control, multi-user ownership and enterprise audit identity.

7. Append-Only Run Event Log

Record run_started, agent_started, agent_completed, artifact_rejected, rollback, code_executed, requirement_changed, evaluation_started and evaluation_completed. Each event has a monotonic sequence number, run_id, event type, status, redacted summary and timestamp. Full prompts are not logged by default.

8. Security Acceptance Criteria

Generated code cannot access the host.

Generated code cannot access the network.

Execution terminates on timeout/resource limits.

Model API keys never appear in generated files.

Invalid artifacts cannot directly control downstream agents.

Run events are append-only for the local execution history.

Authentication/roles are explicitly out of scope for the MVP.

DOCUMENT 4 — FRONTEND SPECIFICATION

1. CLI-First Interface

mas run --task "Build a task management API" --model gpt-4o mas run --file requirement.txt --model gpt-4o-mini mas status RUN_ID mas logs RUN_ID mas results RUN_ID mas change RUN_ID --request "Add task priorities and remove the old status field" mas evaluate --benchmark srdd20 --system mas --model gpt-4o --repeats 3 mas evaluate --benchmark srdd20 --system mas --model gpt-4o-mini --repeats 3 mas ablate --feature router

2. CLI Output

Run: 8f31... Task: Budget Optimizer Model: gpt-4o [01] Manager ✓ [02] Router ✓ score=2 SIMPLE [03] Developer ✓ 4 files [04] AST ✓ cycles=0 [05] Shared test ✓ [06] Ruff ✓ 0 violations [07] Bandit ✓ 0 high [08] Radon ✓ [09] Reviewer ✓ no critical findings [10] Reflection STOP: marginal gain below threshold Tokens: Estimated cost: $... iCoder Quality: 0.xxx Adaptive score: 0.xxx Quality / 1k tokens: 0.xxx MAS pytest diagnostic: pass=xx/xx, coverage=xx%

Numerical values above are formatting examples only, not benchmark results.

3. React Page

One route: `/runs/:runId`. React + Tailwind only; no complex component library.

4. Components

App ■■■ RunHeader ■■■ AgentLog ■ ■■■ AgentEvent ■■■ MetricsTable ■■■ TokenCostChart ■■■ StatusBanner

5. WebSocket Events

ws /ws/runs/{run_id} { "type": "agent.completed", "agent": "ast", "status": "success", "iteration": 1, "timestamp": "..." }

Event types: run.started | agent.started | agent.completed | artifact.invalid | rollback.started | test.completed | metric.updated | run.completed | run.failed

6. UI States

7. Results Table

Columns: Task | System | Model | Repeat | Quality | Completeness | Executability | Consistency | Shared Test | Ruff | Bandit | Radon | Tokens. MAS-only diagnostic columns: pytest pass rate | pytest coverage. Filters: system, model, ablation and task category.

8. Token / Cost Chart

Show total provider-reported tokens and estimated cost. Also expose Quality / 1k tokens. Token count is the primary efficiency measure; cost is derived from recorded provider usage and versioned pricing metadata.

DOCUMENT 5 — FEATURE TICKET LIST

Twenty-five tickets after the requested merges and additions. Each is scoped to one coding session.

Evaluation Configuration

Main comparison

20 SRDD tasks: four per category, fixed random seed.

3 repeats per task.

Systems: MAS and ChatDev.

Model: GPT-4o.

Runs: 20 × 3 × 2 = 120.

Cheaper-model MAS arm

20 same tasks.

3 repeats.

System: MAS only.

Model: GPT-4o-mini.

Runs: 20 × 3 = 60.

Ablations

10 tasks × 2 repeats × 6 configurations = 120 runs.

Model: GPT-4o-mini only.

Configurations: Router OFF; AST OFF; context slicing OFF; cheap reviewer OFF; adaptive stop OFF; always-on UML/Sprint.

Keep task set, seed, model settings and neutral scoring scripts constant.

Core benchmark total = 120 + 60 + 120 = 300 model runs.

Auxiliary diagnostics

Router accuracy: 10 hand-labeled tasks; no additional LLM generation required.

Requirement change: six selected tasks, one injected change per task; measure update success, regression behavior and obsolete-code removal.

Fault injection: F1 malformed UML and F2 incomplete user story; inject into selected workflow executions and record detection/recovery. These are not a new full benchmark matrix.

Token Budget

Measured token usage must come only from provider API usage fields. For planning only, a configurable engineering cap of 40,000 tokens/run for GPT-4o and 20,000 tokens/run for GPT-4o-mini gives:

GPT-4o main: 120 × 40,000 = 4.8M planning tokens GPT-4o-mini MAS: 60 × 20,000 = 1.2M planning tokens GPT-4o-mini ablate: 120 × 20,000 = 2.4M planning tokens ---------- Total planning envelope: 8.4M tokens

These are engineering budget caps, not measured benchmark numbers. Actual usage is reported from provider API usage fields. A one-task pilot may replace the caps with measured condition-specific means before the full batch.

Research Positioning

Recommended claim wording:

“MAS improves over ChatDev by X% on [metric], while addressing the computational-overhead limitation observed in iCoder; iCoder reported a Y% gain over its baselines.”

Do not write: “MAS outperforms iCoder.” The comparison with iCoder is indirect and limited to the paper's reported results.

Published iCoder Reference Results

These values are published reference values, not MAS measurements. The paper reports 3%–7% quality improvements over baselines under GPT-3.5 Turbo in its abstract and describes ablation evidence for iterative development, architecture, reverse dependency modeling and review.

Paper-to-MAS Design Rationale

Limitations

First, most SRDD tasks may route as SIMPLE under the deterministic Router, so the architectural contribution of UML/Sprint stages may be exercised less frequently than in an always-structured system. Second, the comparison with iCoder is indirect: MAS does not reproduce iCoder and uses only the paper's reported gains/results as reference. Third, the task set is small at 20 requirements for the main comparison, so statistical power and generalization are limited. Fourth, consistency uses embedding similarity, which is a weak proxy for true requirement satisfaction and should not be interpreted as a complete semantic correctness measure.

Future Work — list only

Human approval gates

Large multi-module scalability

Multi-language support

Enterprise benchmarks

Authentication and role-based access control

Reference

[1] Yuxuan You, Dejun Ning, Shaohao Cai, Junqi Bai, Jiacheng Zheng, Jiyan Chen. “iCoder: A multi agent software development platform.” Information and Software Technology, Volume 198, 2026, Article 108224. DOI: 10.1016/j.infsof.2026.108224. Key sections used: Sec. 3.1 system architecture; Sec. 3.2 core agent design; Sec. 3.3 artifact collaboration, Sprint decomposition and iterative control; Sec. 4 evaluation; Tables 1–5; Figs. 1–9.

CHANGELOG — Requested Revision

1. Merged MAS-03 + MAS-04 into “Model adapter + token logger”.

2. Merged MAS-09 + MAS-10 into “Changed-context + UML slicing”.

3. Merged MAS-11 + MAS-12 into “AST analyzer + cycle detector”.

4. Added SRDD loader ticket.

5. Added iCoder-metric scorers ticket and named `all-MiniLM-L6-v2` for MAS consistency scoring.

6. Added ChatDev adapter ticket using provider API usage counts for tokens.

7. Added change-request flow ticket.

8. Added fault-injection ticket for F1 malformed UML and F2 incomplete user story.

9. Added ablation-flags ticket; total ticket count is now 25.

10. Removed MAS-generated pytest from the main cross-system quality score.

11. Added the shared fixed neutral test method for both MAS and ChatDev; MAS pytest is now a separate diagnostic.

12. Reduced the main experiment to 20 × 3 × MAS/ChatDev on GPT-4o, plus MAS-only GPT-4o-mini; added 10 × 2 × 6 GPT-4o-mini ablations.

13. Added total core run count = 300 and an explicitly labeled engineering token-budget envelope.

14. Simplified security to local single-user MVP; removed JWT, roles, ownership checks, users and audit_logs.

15. Made Router scoring concrete with deterministic keyword/regex rules, 0–3 SIMPLE / 4+ STRUCTURED thresholds, and a 10-task accuracy check.

16. Defined adaptive stopping weights, STOP_EPSILON=0.02, PATIENCE=2 and MAX_ITERATIONS=3, all configurable in one file.

17. Added reproducibility fields: model/version, temperature, seed, embedding model, library versions and Docker image tag.

18. Specified that token measurements for MAS and ChatDev come only from provider API usage fields.

19. Added the requested limitations paragraph covering SIMPLE routing frequency, indirect iCoder comparison, 20-task size and embedding-based consistency limitations.